FleetCheck Case Study: Candidate Submission

Abid Hasan · Tech Lead · 12 October 2026

Part 1: Understand the scope

Six weeks is enough for a working MVP only if the garage integration and the fairness proof are pinned down in week 1. My questions go to whoever holds those answers first, ordered by how costly a wrong guess is and how long the answer takes. The week-6 target is a **staging demo with a real garage path**, with pilot customers onboarded right after.

a) and b) Five questions, in priority order, with the assumption I build on if there is no answer in a week

#	Question to the client and why it matters	Assumption if no answer in a week
1	**What does the garage SOAP API offer?** WSDL, sandbox, auth, rate limits, sync or async results, and whether an order can be looked up by our reference. It is the only hard external dependency and sets the schedule.	No sandbox until week 2. Poll for results, de-duplicate on vendor result id and version, reconcile a timed-out submit instead of retrying it. Build against a mock made from the WSDL.
2	**What must a regulator verify about the random pick?** Percentage, rounding, cut-off time, who sees the proof. "Prove the pick was fair" decides the algorithm and what we store.	Deterministic seeded draw. Store the frozen eligible list, rule and algorithm version. Picks stay hidden until released. Regulators get an export and a verification script, not a login.
3	**What is an inspection in the product?** Types, who books and schedules, what a result contains, who pays. "Order" could be one button or a negotiation over dates and quotes.	Order = vehicle + type + garage, confirmed by the garage through the API. Result = pass, fail or advisory, with defect lines and an optional certificate. Payment stays outside the platform.
4	**How big is it?** Companies, vehicles per company, users, whether one user can belong to several companies, and what ops may do beyond read. The largest tenant decides where performance work goes.	Tens to low hundreds of companies, the largest about 200,000 vehicles. One company per user. Ops can read everything and take a few audited actions, with no data editing.
5	**What does "must be secure" mean?** Jurisdiction, personal data, region, retention, audit expectations. A wrong region is expensive to move later.	GDPR-grade baseline in one region (EU if the jurisdiction is unknown). Encryption at rest and in transit, MFA for admins and ops, audit log of writes and ops access. Retention stays an open question.

Six-week sequence that follows from the answers

Weeks	Outcome
1	Garage spike with saved real responses; canonical model and adapter interface frozen; go or no-go on the garage API; RLS skeleton and tenant contract tests started
2	Vehicles, CSV import and order flow end to end against the fake garage; jobs outbox, leases and retries
3	Real garage sandbox path, duplicate and timeout handling, ops integration-health view; security baseline (MFA, roles, audit log)
4	Quarterly selection with stored evidence and verification script; ops console read paths and audited actions
5	Hardening: load and garage-failure tests, backups and restore drill, monitoring and alerts; Terraform for staging
6	Staging demo with a real garage path; pilot customers onboarded right after

c) Scope in three lists, one line of reasoning per item

In the 6-week MVP

Item	Why
Company onboarding by ops	Replaces self-serve signup, which needs billing and abuse controls
Sign-in with MFA; roles (company admin, fleet manager, viewer; ops)	"Must be secure"; use a managed identity service
Vehicle register with CSV import	Large fleets cannot onboard by hand
Order inspections, track status, see results	The core product loop
Garage adapter: queue, retries, rate limits, safe duplicates	The main technical risk, so it is real, not mocked
Quarterly random selection with stored audit evidence	A stated requirement, and hard to retrofit
Evidence export and verification script	Cheaper than a regulator portal and still provable
Ops console: cross-company read, integration health, a few audited actions	A stated requirement; read-first keeps risk low
Audit log, monitoring, Terraform, CI/CD, staging	Not features, but nothing is shippable without them

After the MVP

Item	Why
Ops copilot (Part 3)	Not in the core flow; needs clean results data that exists only after launch
Regulator portal	The evidence export covers the legal need first
Email or SMS notifications, scheduling UX, garage comparison	Depend on how the garage API behaves; status in the portal proves the loop
Self-serve signup, billing, payments	Need commercial decisions the brief does not contain
SSO and custom roles	Large customers will ask; not needed to prove the product
Dashboards, scheduled reports, public API and webhooks	Each is a feature set of its own
Public randomness source for the pick	Stronger fairness proof, but an external dependency

Not doing

Item	Why
Changing or replacing the garage SOAP API	Not ours to change; we wrap it in an adapter
Our own identity provider or payment processing	Commodity and high risk, no differentiation
Native mobile apps, telematics, live tracking	A different product; the responsive web portal is enough
Microservices	A modular monolith plus a worker ships faster; module boundaries keep the option open

Part 2: Design the system

A **modular NestJS monolith on PostgreSQL, plus a separate worker**. Each choice is the simplest one that holds up under the three hard requirements: tenant isolation, an unreliable vendor, and a provable random pick.

Data model

Relationships: company → vehicles → inspection orders → result → defects; company → users through memberships; a selection run → its selected vehicles → surprise orders. Garages are global. **Every tenant table also has** id (uuid primary key), company_id NOT NULL, created_at and updated_at; company_id comes first in every index.

Table	Columns	Keys, foreign keys and indexes
Tenants and access		
companies	name, status (active, suspended), region	PK id (the tenant itself; no company_id)
users	idp_subject, email, display_name, kind (customer, ops), ops_role, status	unique(idp_subject); unique(email); global table
memberships	user_id, role (admin, fleet_manager, viewer)	PK(user_id, company_id); unique(user_id) while one company per user
Fleet		
vehicles	vin, registration, make, model, year, vehicle_type, status (active, inactive), active_from, deactivated_at	unique(company_id, vin); unique(company_id, registration) where active; index(company_id, status, id)
vehicle_imports	file_key (S3), status, total_rows, imported_rows, error_report_key, created_by	index(company_id, created_at desc)
Inspections		
garages (global)	vendor_ref, name, region, inspection_types, status	unique(vendor_ref)
inspection_orders	vehicle_id, garage_id, type (routine, surprise), requested_date, status, vendor_ref, selection_run_id, created_by, cancelled_at	FK(company_id, vehicle_id); FK(garage_id); unique(vendor_ref); unique(selection_run_id, vehicle_id); index(company_id, status, created_at desc)
inspection_results	order_id, outcome (pass, fail, advisory), inspected_at, vendor_result_id, version, certificate_key, raw_payload_key	FK(company_id, order_id); unique(vendor_result_id, version); index(company_id, inspected_at desc)
defects	result_id, code, severity, description	FK(company_id, result_id); index(company_id, code)
Integration, selection and audit		
jobs (outbox)	kind, order_id, seq, status, attempts, lease_owner, lease_until, next_attempt_at, last_error, idempotency_key	unique(idempotency_key); unique(order_id, seq); index(status, next_attempt_at)
selection_runs	quarter, percentage_bp, algorithm_version, eligible_count, selected_count, snapshot_sha256, snapshot_key (S3), commitment, seed_ciphertext, released_at	unique(company_id, quarter): one draw per quarter
selection_members	run_id, vehicle_id, score, rank	PK(run_id, vehicle_id); FK(company_id, run_id); FK(company_id, vehicle_id)
audit_log	actor_id, actor_type, action, entity_type, entity_id, request_id, changes (json), at	append-only; partitioned by month; index(company_id, at desc)
idempotency_keys	key, request_hash, response_status, response_body, expires_at	unique(company_id, key)

- Child references are **composite**, e.g. FK(company_id, vehicle_id) → vehicles(company_id, id), so an order cannot point at another company's vehicle. Vehicles are deactivated, never deleted, so history stays intact.

API

- **REST with OpenAPI under /v1.** The company always comes from the verified token, never from the URL or body.
- **Cursor pagination** on lists; an **Idempotency-Key** header is required to create orders. Errors use one JSON problem format with a request id.

Area	Endpoints
Vehicles	GET, POST /vehicles · GET, PATCH /vehicles/{id} · POST /vehicles/{id}/deactivate · POST /vehicles/import (async) · GET /vehicle-imports/{id}
Garages	GET /garages?type=&region= (garages that offer an inspection type)
Inspections	POST /inspections (single or bulk) · GET /inspections?status=&vehicle= · GET /inspections/{id} · POST /inspections/{id}/cancel · GET /inspections/{id}/result
Surprise picks	GET /surprise-selections · GET /surprise-selections/{id} (visible only after released_at)
Company admin	GET /users · POST /users/invite · PATCH /users/{id}/role · DELETE /users/{id} · GET /audit-log

Area	Endpoints
Ops (own prefix, guard and DB role)	GET, POST /ops/companies · POST /ops/companies/{id}/suspend · GET /ops/inspections · GET /ops/vehicles?search= · POST /ops/jobs/{id}/retry · GET /ops/integration/health · GET /ops/selection-runs/{id}/evidence · GET /ops/audit-log

Tenant isolation: Row-Level Security (RLS) plus application guards

Choice: PostgreSQL RLS. Application guards still check authorization first.

- **Why:** the database refuses cross-tenant rows on every path (raw SQL, scripts, a forgotten WHERE). A bug returns zero rows, not another company's data.
- **Setup:** the app connects as a non-owner role without BYPASSRLS; RLS is enabled and forced on every tenant table.
- **Per request:** set_config('app.company_id', <id from token>, true) inside the transaction, so it is safe with pooling. Policy for USING and WITH CHECK: company_id = NULLIF(current_setting('app.company_id', true), '')::uuid.
- **Ops:** a separate database role and pool; cross-company reads, writes only through a few audited functions.
- **Tested:** CI contract tests seed two companies and assert nothing crosses. Checked on PostgreSQL 16: no context returns zero rows, cross-company writes fail.
- **Trade-off:** week-1 plumbing, a transaction per request, and empty results that are harder to debug. Application-only filtering is cheaper, but one missed filter is a silent leak.

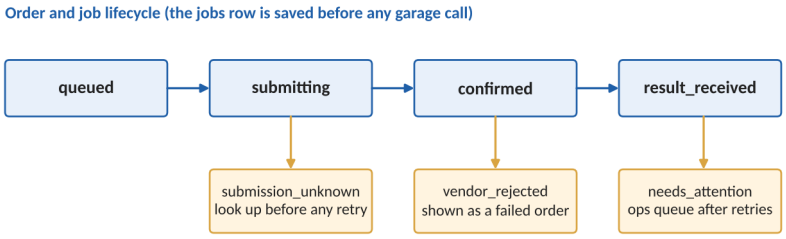
Roles

Role	Can do
Company Admin	Everything in own company: users and roles, vehicles, orders, results, surprise picks
Fleet Manager	Vehicles, orders, results, surprise picks; no user management
Viewer	Read vehicles, results, surprise picks
Ops Support	Read all companies; audited actions only (retry a job, resend a notification)
Ops Admin	Ops Support plus create and suspend companies; never edits results or re-draws a selection

- **Also:** least-privilege service accounts for scheduler and worker; regulators get an evidence export, not a login; break-glass infrastructure access is time-boxed and logged.
- Permissions are checked as **capabilities** (e.g. vehicle:write), so custom roles later do not touch controllers.

Vendor integration: slow, down, duplicate

The user never waits on the garage, and no accepted order is lost. The order and its job are saved in one transaction; only the worker calls the garage, through one adapter that maps SOAP to our own types (XML external entities disabled).

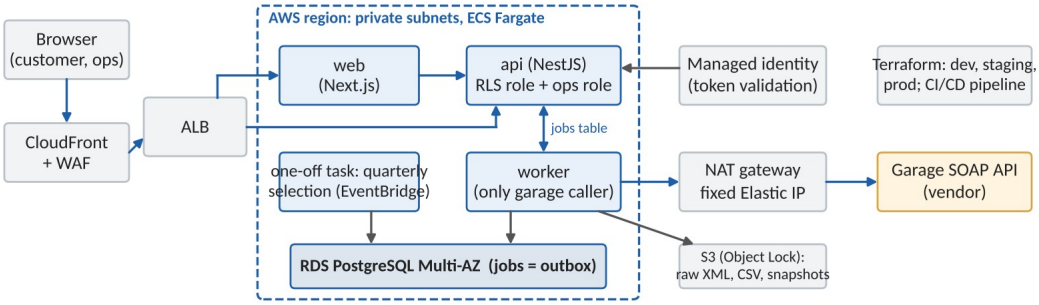


- **Slow:** explicit timeouts and a concurrency limit per garage; the UI shows "waiting for garage".
- **Down:** retries with backoff and jitter, a circuit breaker per endpoint, then the needs_attention queue for ops. **Leases:** a stale worker's completion is rejected.
- **Same result twice:** INSERT … ON CONFLICT (vendor_result_id, version) DO NOTHING. A newer version supersedes; the same version with a different payload is flagged.
- **Timeout on submit:** look the order up by our reference before any retry. If the API cannot, the order goes to submission_unknown and ops decides. Never a blind retry.
- **Fair quotas:** per-company and global limits live in Postgres; dispatch rotates across companies.
- **Visibility:** raw request and response XML saved to S3 with secrets redacted; metrics on latency, errors, circuit state, queue depth and oldest job age.
- **Queue choice:** the jobs table sits behind a small queue interface, so SQS can become the transport later without changing the state model.

Deployment on AWS

- **Edge and compute:** CloudFront and WAF, then an ALB; web (Next.js), api (NestJS) and worker on ECS Fargate in private subnets.
- **Data:** RDS PostgreSQL Multi-AZ with KMS and point-in-time recovery; S3 with Object Lock for raw XML, CSV imports and selection snapshots.

- **Garage traffic:** only the worker calls out, through a NAT gateway with a fixed Elastic IP. **Selection:** EventBridge starts an idempotent one-off task.
- **Identity:** a managed provider chosen with the client; separate realms for ops and customers, MFA for ops.
- **Observability and secrets:** CloudWatch logs, metrics and alarms; traces carry one request id from browser to garage call; Secrets Manager for database and garage credentials.
- **Environments and pipeline:** dev, staging and prod in Terraform; CI runs lint, tests on real Postgres and tenant contract tests, terraform plan on PRs, rolling deploys.



Delivery risk: the garage SOAP API

The vendor, not our code: no WSDL seen yet, a late sandbox, and surprises (undocumented errors, limits, no order lookup) landing in the last weeks. **Week 1:**

- **Day 1:** ask for the WSDL, sandbox, rate limits and a named technical contact; escalate on day 2 if there is no contact.
- **Days 1–3:** a time-boxed spike. Call every operation, save real responses as fixtures, record latency, errors and lookup behavior.
- **Day 3:** freeze our canonical model and adapter interface, so the team builds against a fake garage.
- **End of week 1, go or no-go:** if results cannot be fetched programmatically, ops uploads results and the client chooses between scope and date. A simulator is never a silent substitute.

Part 3: Design an AI feature (ops copilot)

The copilot is a **read-only analyst**. Every fact and calculation comes from tenant-scoped tools; the model only reasons over what they return. No model can be promised never to err, so the design makes unsupported claims detectable and drops them before a user sees them.

Flow: user asks → model calls read-only tools → tools return aggregates with evidence ids → model writes a summary and flags citing those ids → verifier checks every number and id → UI shows the summary, the rule-based flags and links to the records. On any failure, the user sees the rules-only view.

What goes into the prompt and context, and what stays out

- **In:** fixed instructions, the agreed risk rules, tool schemas, and an output schema (summary plus flags citing evidence ids; "unknown" when data is missing).
- **Tools:** read-only and typed (company summary, flagged vehicles, vehicle history, defect statistics). They compute risk signals from the rules: failed or advisory results, repeated defects, overdue inspections, worsening trend.
- **Out:** driver names and contacts (vehicles are opaque references; the UI adds plates), other companies' data, raw vendor XML, secrets, commercial terms, and garage free text unless sanitized and marked untrusted.

Preventing cross-company leaks

- Tools take **no company or user argument**. The server injects them inside the same RLS transaction as the app (Part 2).
- No write tools, no SQL, no web access. Memory and caches are keyed by company and rebuilt per session.
- Only approved providers with no-training terms, in the required region; no fine-tuning on customer data.
- **Test:** two seeded companies with canary strings must never cross, including adversarial prompts and injected garage notes.

Measuring accuracy and catching hallucinations

- A **verifier** checks every number, date and id against the recorded tool results and drops anything unsupported.
- Rule-based flags come straight from the tools. The model can explain them but cannot remove one.
- **Offline evaluation set** (synthetic plus anonymized, ground truth from ops): flag recall and precision, faithfulness, citation validity, canary leaks, injection cases. It runs on every prompt, tool, model or provider change.
- **Shadow mode** before launch, then sampled review and an alert on the verifier-rejection rate.

Cost and latency controls

- Summaries are precomputed when results arrive and cached by company, data version and prompt version.
- Hard caps on tool calls and output length; budgets and rate limits per company and globally; a timeout falls back to the rules-only view.
- **Provider-neutral gateway:** one adapter per provider, routing by cost and latency, automatic failover. A provider must pass the evaluation set before it can take traffic.

What the model must never do unsupervised

- Take any action: order or cancel an inspection, change a status, send a notice, edit a result.
- Make safety or compliance calls. It says "flagged for review because…", never "unsafe".
- Touch the random selection, its seed, pending picks or the audit trail.
- Reach customers before passing the evaluation gate. Customers see only their own company, labelled as an automated summary.

Part 4: Build one piece (Option A, quarterly random selection)

Code: public repository **https://github.com/gitAbid/quarterly-selection**. Option A only.

How it works

- Each draw gets a **fresh random 32-byte seed**. Every vehicle gets a score, HMAC-SHA-256 over (company, quarter, vehicle id). The lowest ceil(count × percentage) scores are picked, so every vehicle has an equal chance.
- **One row per company and quarter**, enforced by the database. A retry returns the stored draw and never draws again. Last quarter's picks stay eligible; each quarter is an independent draw.
- The seed is **encrypted at rest** and revealed only in the audit export. A commitment (a hash of the seed) is stored with the draw, and the verifier checks the seed against it. This proves the draw is internally consistent, not that it was never redone, because the commitment is not published beforehand. The verifier recomputes the selection from the export alone: no database or key needed.

How to run it

- npm ci, then npm test (131 tests in 7 files, about 15 s) and npm run typecheck. Requires Node 22.12+, 24 or 26. The database is embedded SQLite, so there is nothing else to install.
- npm run demo draws 20 vehicles, restarts, retries and verifies an audit export ("ok": true). npm run benchmark times 20 and 200,000 vehicles. The README has the CLI commands.

Result: 131 tests pass and the typecheck is clean (verified on a fresh clone with Node 26).

Complexity

Time O(n log n): one HMAC per vehicle, then a sort. **Space O(n):** the eligible list is stored so the draw can be replayed. Measured at 200,000 vehicles: creating a draw took **1.2 s on Node 22** and **4.2 s on Node 24** (single run, arm64 laptop).

Edge case the brief does not mention

The fleet changes between the first draw and a retry, for example a vehicle is added after the draw. The service returns the original draw, never redraws, and sets inputsMatch: false so the caller sees the mismatch. The draw is not silently replaced. The code reports a flag, not a list of differences.

Design choices

- **Rounding up:** any non-empty fleet gets at least one pick (7 vehicles at 25% selects 2).
- **Duplicate vehicle ids are rejected** before any draw.

Left for later

Listed in the README: draws for future quarters are accepted; the seed commitment is not published before the quarter; every read re-verifies the draw, which costs up to about 2 s at 200,000 vehicles; key backup and rotation; SQLite is single-machine, and production would put the same store interface on PostgreSQL.

Part 5: My AI log

Tools. My own reading and design first, then Claude (chat and Claude Code) and Codex. Codex did the Part 4 analysis, research and first plan; Claude cross-checked and reviewed it; Claude Code finalized the plan. Claude also did the final formatting, requirements check and simplification of this document. I made the decisions; the AI produced options and drafts.

How I worked

- **Read the case myself first.** I worked out my own solution direction for each part before opening any AI tool.
- **Independent analyses.** Claude and Codex ran deeper breakdowns separately, so I had something to compare against my own work.
- **Per part:** the models split each part into decisions, raised questions, and listed pros and cons per option. I compared, weighted and mapped them to what fits a 6-week MVP.
- **Converge.** Revised with Claude, then a consistency pass across decisions; went back and forth under scope, risk and simplicity until the draft converged.
- **Decide section by section** through a decision form, overriding the AI where I disagreed.
- **Final pass with Claude:** formatting, a check against every requirement in the brief, and simplifying the language. My own final review came last.

Part	Tools	What I did	Outcome
1. Scope	Own reading, Claude, Codex	Own questions and scope; both models broke the brief into questions, assumptions and pros and cons; compared and mapped to the MVP	Third-party-first question order; copilot after MVP; CSV import in MVP; staging demo then pilot
2. System design	Own design, Claude, Codex	Compared isolation, queue, ops access and identity options; ran RLS behavior checks on real PostgreSQL 16	RLS plus guards; jobs as outbox with leases behind a queue interface; separate ops role; identity left to client
3. AI feature	Claude, my redesign	Reviewed the AI's design against my requirement: facts deterministic, analysis useful	Tool-based agent with scoped read-only tools; provider-neutral
4. Code	Codex, Claude, Claude Code; my review	Codex: requirements analysis, research and a plan. Compared its results with Claude and reviewed them with Claude, then finalized the plan in Claude Code. Test criteria first; test-driven build; my own code review; manual sanity check. After the first build, Claude Code refactored the code (SOLID design, strict typing, layered folders, formatting). The existing tests kept behavior unchanged.	Option A, quarterly random selection

Accepted as-is: the API design. REST, tenant taken from the token, idempotency key on order creation, cursor pagination, separate ops routes. It matched the design I had worked out on my own and agreed with the other analyses, so it was corroborated by independent routes, not just plausible.

Rejected and heavily changed: the first copilot design. The AI had rules compute the flags while the model only explained them. The copilot must be deterministic about the data and still useful in its analysis, so in my version every fact comes through read-only, scoped tools and the model analyzes what they return. I kept the verifier that drops unsupported claims and the flags shown straight from tools. I also replaced "no hallucinations" with "any unsupported claim is caught before a user sees it".

How I verified the result. Part 4: Codex analysis and plan, cross-checked and reviewed with Claude, plan finalized in Claude Code; then test criteria before implementation, a test-first build, my own code review and a manual run. The refactor was checked by re-running the same tests and a fresh-clone install. Design decisions: cross-checked against my own design and two AI analyses, then a consistency pass. Tenant isolation: RLS behavior checked on a real PostgreSQL 16 rather than trusting the description. It caught a real flaw: after a transaction-local setting ends it reads as an empty string, so the uuid cast failed instead of returning no rows. The fix is NULLIF. **Not verified:** the vendor's real behavior, model quality, and RLS performance on a very large tenant. These are design intent, not measured results.

Five rules for a six-person team using AI on this client's code

1	**A human owns every line:** AI code gets normal review, and the author can explain it.
2	**No client data, secrets or personal data in prompts,** except in client-approved tools with no-training terms.
3	**Small, reviewable, tested changes:** no large, unexplained AI-generated pull requests.
4	**Respect architecture and security boundaries:** changes to auth, data models, infrastructure or public APIs need explicit approval.
5	**Verify before relying:** run it or read the primary docs; check every suggested dependency exists, is maintained and licensed.

Part 6: How I work

1. News they did not want to hear

The team had committed to a Java 6 to Java 21 upgrade. During assessment I found compatibility issues that would have caused rework later. I told the PM and the GM: **"Before proceeding with the full upgrade, we should validate the compatibility issues. Otherwise, we risk spending more time fixing unexpected problems later."** I proposed validating the migration on a smaller scope first, with AI-assisted tools for the repetitive work and proper testing and review. That approach was taken, and it brought the estimated effort down from around 80 hours to 36.

2. A promised deadline that became at risk

The team had committed to integrating a telco's IN APIs for charging, refunds and reversals, built against the provided documentation and WSDL. As soon as sandbox access arrived I tested against it and found the documentation was outdated. I told the PM and the GM straight away: **"The API doesn't match the documentation we implemented against. We need the correct specifications before proceeding, otherwise we'll waste more time reworking the integration."** The correct specifications were requested and provided. Because the integration used an interface-based design with a separate adapter layer, the core business logic stayed unchanged; only the vendor connector was updated.

3. The work I am proudest of

I helped take a telecom platform's throughput from around **250 TPS to around 800 TPS**, measured with performance tests before and after. My part was investigating bottlenecks with thread dumps and memory dumps to find thread contention and memory problems, then implementing: query optimization; asynchronous, buffered logging to cut I/O; a normalized table design so flows no longer locked the same table; connection- and thread-pool tuning; async and batch processing; and memory optimization of the cache stores. The team supported integration, testing and regression validation. What I would do differently: set performance baselines earlier (latency, resource use, error rates) and load-test in a fully production-equivalent environment.

4. An engineering habit I keep under deadline pressure

I do not trade testing or code quality for a date. I break the work into manageable goals and keep the team aligned on priorities, progress and blockers. During the 250 to 800 TPS work there was pressure to release; I did not rush it, and I worked with the team to validate performance in an environment close to production. **A teammate sees it** in stand-ups, in pull request reviews and in day-to-day work: I keep everyone aligned, so pressure is shared, changes are validated, and delivery is reliable.

Terms

RLS: Row-Level Security, a database rule that hides rows from other tenants. **Outbox:** a table written in the same transaction as the business change, read by a worker to do the external call. **Lease:** a time-limited claim on a job, so a stalled worker can be replaced. **Idempotent:** safe to repeat; the second attempt changes nothing. **Circuit breaker:** stops calling a failing endpoint for a while. **Object Lock:** S3 storage that cannot be altered or deleted before a set date.